



ParKing

**Jhon Freddy Patiño Meza
Nicolás Aguirre Velásquez**

Índice

1

Arquitectura

2

Desafíos técnicos

3

Expectativas de mejora

4

¿Qué aprendimos?

5

Reflexión

6



Frontend — Vue 3

Vistas

Públicas

LoginRegister

LotListMap

Admin

ConfigRates

DiscountsStaff

Reports

Staff

EntryExitInvoices

Customer

Vehicles

Router + Guards

Vue Router 4

guards.js

routes/rol

Estado (Pinia)

authStore

API (Axios)

http.js

auth

parkingLots

entries

rates

discounts

staff

invoices

reports

vehicles

Componentes

AppTopBar

AppSidebar

ParkingGrid

CellTile

REST
↕
JSON

Backend — Spring Boot

Security

.security — JWT / autenticación

JwtAuthFilterJwtTokenProvider

Config

.config — configuración (Security)

SecurityConfig

Exception

.exception — manejo global de errores

GlobalExceptionHandlerErrorResponse

Controller

.controller — endpoints REST

AuthControllerParkingLotControllerStaffControllerVehicleControllerInvoiceControllerCustomerController

DTO

.dto — objetos entrada/salida de la API

VehicleEntryRequestVehicleExitRequestVehicleExitResponseActiveEntryResponseParkingMapResponseCellDtoInvoiceResponseAuthResponseParkingLotRequestStaffRequest

Service

.service — lógica de negocio

AuthServiceParkingLotServiceCellServiceEntryRecordServiceRateServiceDiscountConfigServiceRealDiscountCalculatorReportService

Repository

.repository — acceso a BD (Spring Data JPA)

UserRepositoryVehicleRepositoryVehicleTypeRepositoryParkingLotRepositoryCellRepositoryEntryRecordRepositoryPaymentRepositoryInvoiceRepositoryRateRepositoryDiscountConfigRepository

Entity

.entity — entidades JPA / tablas de BD

UserRoleVehicleVehicleTypeParkingLotCellEntryRecordPaymentInvoiceRateDiscountConfig

SQL
↕
JPA

Infraestructura

PostgreSQL 16

Base de datos

Docker

Contenedores

JPA / Hibernate

ORM

Spring Boot

Tomcat embebido

SHA-256

Hash contraseñas

Arquitectura

Capa	Paquete	Ro1
Controller (presentación)	<code>.controller</code>	Endpoints REST
DTO	<code>.dto</code>	Objetos de entrada/salida de la API
Service (negocio)	<code>.service</code>	Lógica de negocio
Repository (datos)	<code>.repository</code>	Acceso a BD (Spring Data JPA)
Entity (modelo)	<code>.entity</code>	Entidades JPA (mapeo a tablas)
Config	<code>.config</code>	Configuración (Security)
Exception	<code>.exception</code>	Manejo global de errores
Security	<code>.security</code>	JWT / autenticación

— 0-0	— 0-1	— 0-2	— 0-3	— 0-4	— 0-5	— 0-6	bicycle 0-7
car 1-0	car 1-1	car 1-2	car 1-3	car 1-4	— 1-5	bicycle 1-6	bicycle 1-7
— 2-0	— 2-1	— 2-2	— 2-3	— 2-4	— 2-5	— 2-6	motorcycle 2-7
car 3-0	car 3-1	car 3-2	car 3-3	car 3-4	— 3-5	motorcycle 3-6	motorcycle 3-7
— 4-0	— 4-1	— 4-2	— 4-3	— 4-4	— 4-5	— 4-6	motorcycle 4-7
motorcycle 5-0	motorcycle 5-1	motorcycle 5-2	motorcycle 5-3	motorcycle 5-4	motorcycle 5-5	motorcycle 5-6	motorcycle 5-7

Desafío: El mapa del parqueadero

¿Cómo representar físicamente un parqueadero con espacios de diferente tipo (parqueo/tránsito), diferente vehículo (carro/moto/bici) y estado en tiempo real (libre/ocupado)?

La solución

Backend: cada celda es un registro en la tabla cell con campos row, col, code, cellType, status, vehicleTypeId. El servicio construye una `List<List<CellDto>>` — una grilla 2D — que el controller entrega al frontend.

Frontend: la grilla se renderiza con `display: grid` en CSS, ajustando dinámicamente el número de columnas según las dimensiones del parqueadero.

Lo que se implementó:

- Vue 3 Composition API y <script setup>
- Reactividad con ref(), computed(), watch()
- Manejo de estado global con Pinia
- Navegación con guards dinámicos en Vue Router
- Comunicación asíncrona con Axios, interceptores y manejo centralizado de errores
- Renderizar una grilla dinámica con CSS Grid sin librerías

Desafío: El frontend

Ninguno del equipo tenía experiencia con Vue 3, Pinia ni el ecosistema moderno de JavaScript para frontend. El backend estaba relativamente más cómodo por ser Java, pero el frontend era territorio completamente nuevo.

Expectativas de mejora

Tarifa plena personalizable

- Ahora mismo está fijada en 6 horas, sería ideal que cada parqueadero la pueda configurar a gusto.

Automatizar aún más la ejecución de los test, e implementar más.

- Actualmente se corre todo manualmente o usando scripts.

Reportes más detallados

- Poder incluir comparativas entre periodos o desglose por cada trabajador.

Mejorar la seguridad del login

- Con fines educativos se usó SHA256, se podría usar un algoritmo más fuerte para encriptar contraseñas.
- Añadir el uso de códigos OTP u OAuth.

¿Qué aprendimos?

Desarrollar software no es, en efecto, solo escribir código

También implica:

- Diseñar arquitectura
- Validar requisitos
- Documentar
- Mantener consistencia
- Probar

Documentar las decisiones es clave

Ayuda a:

- Mantener consistencia entre código y decisiones de arquitectura.
- Mantener el alcance de proyecto.



¿Qué aprendimos?

Es mejor repartir por dependencias en vez de por “quien tiene tiempo”.

- Aprendimos que organizar el trabajo según qué módulos dependen de otros reduce bloqueos y facilita la integración del proyecto.



Aunque la IA sea una herramienta muy buena, también comete errores.

- Nos ayudó a acelerar el desarrollo, pero comprobamos que sus respuestas deben revisarse y validarse antes de incorporarlas al proyecto.



Reflexión

Lo abordado

El curso fue muy completo

- Realmente estamos satisfechos con todo lo abordado en el curso y el desarrollo de este a pesar del paro.

Limitaciones y problemas

Fokin paro

- Hizo que se perdiera un poco el hilo del curso y gran parte de la organización que se llevaba inicialmente en nuestro grupo.

Coordinación del equipo

- Los horarios de clases de los integrantes dificultaban reunirse constantemente.

¡Muchas gracias!

